

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“Jnana Sangama”, Belgaum 590014, KARNATAKA, INDIA



Project Report

On

“RAIDER – Reinforcement Agents for Intelligent Discovery and Exploit Research”

Submitted in Partially fulfillment of the requirement for the award of degree Of

**Bachelor of Engineering In
Artificial Intelligence and Machine Learning**

Of Visvesvaraya Technological University, Belgaum.

Submitted by:

ADEEBA SIDDIQUIE(1AM22AI003)

AYUSH SHREYAS GAYAKWAD (1AM22AI007)

BIBI MARIAM (1AM22AI010)

MURTAZA SAHERWALA (1AM22AI030)

Under the Guidance of:

DR. MEENA DESHPANDE

HOD, Dept. of AIML



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE
LEARNING**

AMC ENGINEERING COLLEGE

18th K.M. Bannerghatta Main Road, Bengaluru – 560083

AMC Engineering College

Bengaluru– 560083

Department of Artificial Intelligence and Machine Learning



CERTIFICATE

Certified that the Project work entitled “**RAIDER – REINFORCEMENT AGENTS FOR INTELLIGENT DISCOVERY AND EXPLOIT RESEARCH**” carried out by bonafide students, **ADEEBA SIDDIQUIE (1AM22AI003)**, **AYUSH SHREYAS GAYAKWAD (1AM22AI007)**, **BIBI MARIAM (1AM22AI010)**, **MURTAZA SAHERWALA (1AM22AI030)** of AMC Engineering College, in partial fulfillment for the award of Bachelor of Engineering in Artificial Intelligence & Machine Learning of Visvesvaraya Technological University, Belgaum during the year 2025-2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The Project report has been approved as it satisfies the academic requirements in respect of Project Work prescribed for the said Bachelor of Engineering degree.

Project Guide
Dr. Meena Deshpande
HOD
Dept. of AIML

HOD
Dr. Meena Deshpande
Associate Professor
Dept. of AIML

Principal
Dr. Yuvaraju B N
Principal,
AMCEC

External Viva

External Name

Signature with Date

1.

2.

DECLARATION

We, **Adeeba Siddiquie, Ayush Shreyas Gayakwad, Bibi Mariam, Murtaza Shabbir Saherwala** the undersigned students of 7th semester Department of Artificial Intelligence & Machine Learning, AMC Engineering College, declare that our project work entitled **RAIDER – Reinforcement Agents for Intelligent Discovery and Exploit Research** is a bonafide work of ours. Our project is neither a copy nor by means a modification of any other engineering project. We also declare that this project was not entitled for submission to any other university in the past and shall remain the only submission made and will not be submitted by us to any other university in the future.

Name	USN	Signature
ADEEBA SIQQIQUIE	1AM22AI003	_____
AYUSH SHREYAS GAYAKWAD	1AM22AI007	_____
BIBI MARIAM	1AM22AI010	_____
MURTAZA SHABBIR SAHERWALA	1AM22AI030	_____



Department of Artificial Intelligence and Machine Learning

VISION AND MISSION OF THE DEPARTMENT

Vision of the Department

To be a premier Department in the field of Artificial Intelligence and Machine Learning to meet the future needs of the Society.

Mission of the Department

1. To provide state of art infrastructure, academic excellence facilities and research with innovative ICT in AI & ML.
2. To provide hands on experience in the advanced tools and technologies in the field of expert systems and Neural Networks.
3. To collaborate Industry Institute Interaction and prepare students ready for the next generation technological needs of the society.
4. To mould students into professionally competent, socially responsible and ethically correct AI & ML Professional.

Program Outcomes (POs)

PO1 - Engineering Knowledge:

Acquire knowledge on fundamentals of Artificial Intelligence and machine learning Engineering, Sciences, Mathematics and Computational aspects.

PO2 - Problem Analysis:

Identify formulate and solve complex AI&ML Engineering problems by choosing and applying appropriate analysis and modeling methods.

PO3 - Design/Development of solutions:

Design and conduct hardware, software, simulation component or process to meet desired needs with realistic constraints including health, safety, cultural, societal and environmental considerations.

PO4 - Conduct Investigations of Complex problems:

Carry out research, experiments, team projects and publish the outcomes in various symposia, conferences and journals.

PO5 - Modern Tool Usage:

Use advanced tools like Pandorabots, Program O, github to solve complex electronics and interdisciplinary problems.

PO6 - The Engineer and Society:

Demonstrate the understanding of impact of engineering solutions on the society and also will be aware of contemporary issues.

PO7 - Environment and Sustainability:

Evaluate the impact of engineering solutions in diverse engineering fields such as health, environment and safety in global and societal contexts.

PO8 - Ethics:

Demonstrate knowledge of professional and ethical responsibilities while presenting sustainable solutions.

PO9 - Individual and Team Work:

Work effectively as an individual and as a leader in interdisciplinary environments.

PO10 -Communication:

Communicate effectively in both verbal and written form.

PO11 -Project Management and Finance:

Apply managerial principles to his/her own work including financial implications and to manage project in multidisciplinary environments.

PO12 -Life-Long Learning:

Participate and succeed in competitive examinations, develop confidence for self- education and ability for life-long learning.

Program Specific Outcomes (PSOs)

PSO1 Ability of applying soft computing techniques, Deep Learning, Data Structures, Computational Intelligence Algorithms, Expert Systems and Neural Networks

PSO2 Software skills and ability of developing advanced training modules using machine learning techniques and Evolutionary Algorithms.

Program Educational Objective (PEOs)

PEO1: Graduate acquires advanced knowledge and skills of AI & ML and develop leadership skills to drive the society.

PEO2: Graduates of program will apply AI & ML tool in real time problems for enhancing knowledge in the inter disciplinary research and startups.

PEO3: Graduate adopt value-based proficiency in solving expert level problems.

ACKNOWLEDGEMENT

First and foremost we would like to thank GOD, the Almighty for being so merciful on us.

We have a great pleasure in expressing our deep sense of gratitude to founder **Chairman Dr. K.R. Paramahamsa** and **Executive Vice President Mr. Rahul Kalluri** for having provided us with a great infrastructure and well-furnished labs for successful completion of our Project.

We express our sincere thanks and gratitude to our Principal **Dr. Yuvaraju B N** for providing us all the necessary support successful completion of our project.

We would like to extend our special thanks **Dr. Meena Deshpande**, Associate Professor and HOD, Department of AIML, for her support and encouragement and suggestions given to us in the course of our project work.

We are grateful to our Project Guide, **Dr. Meena Deshpande**, Associate Professor, Department of AIML, AMC Engineering College, Bengaluru for her constant motivation & timely help, encouragement and suggestion.

Last but not the least, we wish to thank all the teaching & non-teaching staffs of department of Artificial Intelligence & Machine Learning, for their support, patience and endurance shown during the preparation of this project report.

ADEEBA SIDDIQUIE

1AM22AI003

AYUSH SHREYAS GAYAKWAD

1AM22AI007

BIBI MARIAM

1AM22AI010

MURTAZA SHABBIR SAHERWALA

1AM22AI030

ABSTRACT

This project presents **RAIDER**, an autonomous red-team framework composed of specialized Large Language Model (LLM)–driven agents integrated with reinforcement learning and blackboard-based multi-agent coordination. Designed to replicate the collaborative intelligence and decision-making process of human penetration testers, RAIDER consists of dedicated Reconnaissance, Exploitation, Coordination, and Reporting agents that operate together to discover, analyze, and exploit vulnerabilities in complex software systems.

Unlike traditional automated scanners that rely on static signatures or limited rule-based checks, RAIDER dynamically investigates the target environment, performs contextual reasoning through LLM assistance, and executes adaptive multi-step attack strategies such as SQL Injection and Cross-Site Scripting (XSS). The system incorporates a Reinforcement Learning core that optimizes attack sequences over multiple episodes, allowing the agent to learn effective exploitation paths rather than executing predefined payload lists. A shared blackboard architecture further enables inter-agent communication, real-time intelligence sharing, and coordinated decision-making.

The framework also supports live targets and simulated environments, integrates Selenium for real-time DOM analysis, and provides a Human-in-the-Loop interactive dashboard that allows operators to supervise, pause, resume, or terminate ongoing attacks. Through this combination of multi-agent collaboration, context-aware reasoning, dynamic payload generation, and autonomous learning, RAIDER aims to bridge a critical gap in the cybersecurity landscape—making advanced, human level penetration testing scalable, efficient, and accessible for organizations of all sizes.

CONTENTS

CERTIFICATE	i
DECLARATION	ii
VISION AND MISSION OF THE DEPARTMENT	iii
ACKNOWLEDGEMENT	v
ABSTRACT	vi

CHAPTER	TITLE	PAGE. NO
1	INTRODUCTION	1-7
	1.1 Overview	1
	1.2 Objectives	2
	1.3 Methodology	3
	1.4 Purpose, Scope and Applicability	5
	1.5 Organization of the Report	7
2	LITERATURE SURVEY	8-14
	2.1 Introduction	8
	2.2 Summary of Literature Survey	9
	2.3 Drawbacks of Existing Systems	11
	2.4 Problem Statement	11
	2.5 Proposed Solutions	12
3	REQUIREMENT ENGINEERING	15-18
	3.1 Hardware and Software Tools Used	15
	3.1.1 Hardware Tools Used	15
	3.1.2 Software Tools Used	15
	3.2 Software Requirement Specification	16
	3.3.1 Functional Requirements	16
	3.3.2 Non-Functional Requirements	17
4	DETAILED DESIGN	19-28
	4.1 System Architecture	19

	4.2	Data Flow Diagram	22
	4.3	Algorithm Design	23
	4.4	Sequence Design	27
5		IMPLEMENTATIONS	29-31
	5.1	Overview of the System	29
	5.2	Frontend Development	30
	5.3	Backend Development	31
	5.4	Code	32
6		MAPPING TO SDG	33-34
7		RESULTS	35-37
	7.1	Snapshots	35
	7.2	Results	35
	7.2	Evaluation and Testing	36
8		CONCLUSION	38-42
	8.1	Conclusion	38
	8.2	Applications	39
	8.3	Future Enhancements	40
		REFERENCES	43-44

LIST OF FIGURES

Fig. 4.1 RAIDER Backbone

Fig. 4.2 Workflow for Autonomous Red Teaming and Vulnerability Exploitation

Fig. 4.3 Sequence diagram of the Red Teaming Backbone

Fig. 5.1 Code Snippets

Fig. 7.1 Command Centre

Fig. 7.2 Training and Efficiency Graphs

Fig. 7.3 Heatmap of the RL Weights

LIST OF TABLES

Table 3.1 Hardware Requirements

Table 3.2 Software Requirements

CHAPTER 1

INTRODUCTION

1.1 Overview

The rapid evolution of web technologies and cloud-based applications has significantly expanded the digital attack surface. As organizations increasingly rely on online platforms, the frequency and sophistication of cyberattacks—such as SQL Injection, Cross-Site Scripting (XSS), and authentication bypass—have grown substantially. Traditional penetration testing methodologies, although effective, are often manual, time-intensive, and highly dependent on the expertise of the tester. Moreover, conventional automated scanners rely mostly on predefined payload lists and static signatures, making them ineffective against dynamic, modern web applications that require contextual reasoning and adaptive attack strategies. To address these limitations, the cybersecurity landscape is shifting toward intelligent, autonomous red-teaming systems capable of learning, adapting, and reasoning in real time. Reinforcement learning, multi-agent systems, and large-language-model (LLM)–powered analysis have emerged as promising approaches for creating next-generation offensive security tools that behave more like human penetration testers. This project presents **RAIDER**, an AI-driven, multi-agent autonomous penetration testing framework designed to perform intelligent reconnaissance, exploit research, and adaptive attack execution. RAIDER integrates multiple advanced technologies, including:

- A Reinforcement Learning (Q-Learning)–based Coordination Core that optimizes attack decisions.
- Autonomous Recon and Exploit Agents communicating via a shared Blackboard architecture.
- Google Gemini–powered Context-Awareness for understanding webpage structure and DOM elements.
- Automated SQL Injection and XSS exploitation with dynamically generated payloads.
- Selenium-based real-time interaction with live web targets and mock environments.
- A graphical dashboard enabling real-time mission monitoring, human-in-the-loop control, and attack supervision.

Unlike conventional tools that rely solely on static payload dictionaries, RAIDER “researches” the target by analyzing the webpage layout, navigation flow, and input fields before generating exploitation hypotheses. This mimics the reasoning process of human red-team experts. The

system further enhances efficiency through a learning loop where successful or failed attacks continuously refine the model's strategy using RL feedback. With the continuous rise of cyber threats and the need for rapid, accurate, and intelligent security assessments, RAIDER aims to bridge the gap between manual penetration testing and automated scanners. By combining multi-agent collaboration, machine learning, and LLM reasoning, RAIDER offers a next-generation, adaptive, and context-aware offensive security solution for modern web applications. AI agents that automatically reply to YouTube comments can help automate engagement and save time for content creators and businesses. They can analyze comments, suggest responses, and even personalize replies based on the commenter's tone and style.

1.2 Objectives

- To develop an autonomous red team composed of specialized agents with complementary capabilities (LLM-based, RL-based, and tool-driven).
 - * This objective focuses on designing a coordinated set of AI-driven agents, each equipped with distinct strengths suited to its role. The Exploit Agent leverages large language model (LLM) reasoning to analyze web application contexts, interpret DOM structures, and generate intelligent, adaptive payloads. The Coordination Core uses reinforcement learning (RL) to plan actions, optimize attack sequences, and improve decision-making through experience. The Recon Agent operates using industry-standard tools like Nmap to gather accurate environmental intelligence. Together, these heterogeneous agents collaborate to mimic the workflow of human red teams, achieving a balance of data-driven reasoning, learned strategy, and practical reconnaissance capability.
- To build a Recon Agent and an Exploit Agent that perform reconnaissance and gain initial access to the system, respectively.
 - * The Recon Agent is responsible for scanning the target environment, identifying open ports, web services, technologies in use, and potential entry points. The Exploit Agent builds upon the findings of the Recon Agent to generate and execute context-aware payloads for attacks such as SQL Injection and Cross-Site Scripting (XSS). Together, these agents replicate the early stages of a penetration test, where attackers first gather intelligence and then attempt to compromise the system. Their coordinated operation forms the backbone of RAIDER's autonomous attack pipeline.
- To create a system that collaboratively discovers and exploits vulnerabilities in

complex software systems.

* RAIDER aims to introduce a collaborative multi-agent architecture where different agents share information through a centralized Blackboard system. This shared knowledge base allows agents to analyze findings, refine attack strategies, and coordinate actions without explicit human guidance. As a result, the system can identify vulnerabilities, validate them, and escalate attacks through sequential phases—mirroring how professional red teams collaborate during real-world engagements.

- To build a system that moves far beyond the current state of the art by planning and executing multi-step attack paths, similar to a human red team.

* Unlike traditional vulnerability scanners that rely on static payloads and shallow testing methods, RAIDER integrates reinforcement learning (RL) and LLM reasoning to construct multi-step attack chains. The RL-based Coordinator allows the system to learn from previous attempts, adapt strategies dynamically, and autonomously decide the optimal sequence of actions. This empowers RAIDER to execute complex, chained attacks (e.g., recon → login bypass → DOM analysis → XSS escalation) in a manner analogous to highly skilled human attackers.

- To democratize access to advanced security audits, allowing smaller organizations to achieve a higher level of security assurance.

* Security audits conducted by expert red teams are expensive and often inaccessible to small and mid-sized organizations. By automating much of the attack planning, reconnaissance, and exploitation process, RAIDER seeks to provide an affordable, intelligent, and scalable security assessment tool. The system reduces dependence on expensive human expertise while still delivering robust, high-quality penetration testing capabilities—ultimately making advanced cybersecurity more accessible across industries and organizational sizes.

1.3 Methodology

- The methodology adopted for the development of RAIDER involves a systematic, multi-stage approach that integrates reinforcement learning, multi-agent collaboration, and LLM-powered contextual reasoning to create an autonomous red-teaming system. The process is designed to replicate the workflow of professional penetration testers while leveraging AI-driven automation for efficiency, adaptability, and intelligence. The key methodological stages are described below.

1. System Architecture Design

- The first phase involves designing a modular, scalable system architecture capable of supporting autonomous agents and intelligent decision-making. RAIDER is built using a Blackboard-based multi-agent architecture, where each agent operates independently but shares intelligence through a centralized communication system. The architecture includes a Reinforcement Learning Coordinator, Recon Agent, Exploit Agent, LLM Context Engine, GUI Dashboard, and supporting modules such as logging and reporting.

2. Development of Autonomous Agents

- This stage focuses on implementing specialized agents responsible for different phases of the penetration testing process:
- **Recon Agent:** Performs network scanning using tools like Nmap to identify open ports, services, and possible attack surfaces.
- **Exploit Agent:** Utilizes Selenium-driven interactions and LLM-guided DOM understanding to execute SQL Injection, XSS, and other attack payloads.
- Each agent is designed to operate asynchronously, contribute findings to the Blackboard, and respond to instructions from the Reinforcement Learning core.

3. Integration of Context-Aware Reasoning Using LLMs

- To overcome the limitations of static payload-based scanners, the system incorporates Google Gemini for contextual DOM analysis and intelligent navigation. This component extracts webpage structure, interprets form fields, identifies clickable elements, and generates meaningful exploit strategies. By integrating LLM reasoning, RAIDER moves beyond brute-force methods and performs "exploit research" similar to human red teamers.

4. Reinforcement Learning-Based Decision Making

- A Q-Learning-based Coordination Core is developed to optimize attack strategies over time. The RL model:
- Identifies states based on recon results, DOM context, and previous outcomes
- Selects actions such as navigation, payload testing, or escalation
- Receives rewards for successful exploits and penalties for failed attempts
- Updates the Q-table to improve future decision-making
- This learning loop enables RAIDER to autonomously refine its attack sequences across episodes.

5..Multi-Step Exploitation Pipeline

- The methodology includes creating a structured attack pipeline where the system performs:
 - Reconnaissance
 - Target research
 - Payload generation
 - Exploit execution
 - Validation and escalation
- Unlike traditional tools, RAIDER continues attempting multiple attacks even after achieving partial success, enabling deeper penetration into the system.

6. Graphical Dashboard & Human-in-the-Loop Control

- A user-friendly GUI dashboard is developed to provide real-time mission insights, including logs, attack timelines, agent activity, and vulnerability detections. This module enables operators to pause, resume, or terminate the mission, ensuring safe and ethical operation while maintaining transparency in automated attack processes.

7. Testing, Evaluation, and Reporting

- The final stage involves rigorous testing on:
 - Live target websites
 - Controlled environments using `mock_target.py`
- The system generates automated reports summarizing vulnerabilities found, payloads used, timestamps, and mission outcomes. Tools such as `inspect_brain.py` are used to visualize RL performance and Q-table evolution.

1.4 Purpose, Scope and Applicability

Purpose:

The primary purpose of the RAIDER project is to develop an intelligent, autonomous penetration testing framework that replicates and enhances the capabilities of a human red team. As modern web applications grow more dynamic and interconnected, traditional security assessment methods—whether manual or tool-based—struggle to keep pace with evolving attack patterns. RAIDER addresses this challenge by integrating reinforcement learning, multi-agent collaboration, and LLM-driven contextual reasoning to create a system that can independently analyze, plan, and execute sophisticated cyberattacks.

The goal is not merely to automate scanning or payload execution, but to build a system capable of understanding its target environment, researching potential weaknesses, and strategically exploiting identified vulnerabilities. Through its specialized agents, RAIDER performs reconnaissance, analyzes web structures, generates adaptive payloads, and validates discovered vulnerabilities—ensuring a comprehensive and intelligent testing process. The system’s Blackboard-based communication enables seamless coordination between agents, while the graphical dashboard provides transparency and human oversight for safe and ethical operation. By enabling autonomous multi-step attack planning and execution, RAIDER aims to significantly reduce the manual effort required for penetration testing and vulnerability assessment.

Additionally, the project seeks to democratize access to advanced red-teaming capabilities, ensuring that smaller organizations, academic institutions, and resource- constrained teams can still maintain strong security postures. Ultimately, RAIDER’s purpose is to deliver an efficient, scalable, and intelligent cybersecurity auditing solution that elevates both the effectiveness and accessibility of modern security assessments.

Scope and Applicability:

RAIDER is designed as a next-generation autonomous red-teaming framework focused primarily on assessing the security of modern web applications and network-exposed services. Its core functionality includes reconnaissance, intelligent exploit research, adaptive attack execution, and automated vulnerability reporting. The system leverages reinforcement learning, multi-agent collaboration, and LLM-powered contextual analysis to discover and exploit weaknesses in complex digital environments.

RAIDER’s applicability extends across multiple security domains. It can perform comprehensive reconnaissance, identify attack surfaces, analyze web page structures, and autonomously execute multi-stage attacks such as SQL Injection (SQLi) and Cross-Site Scripting (XSS). By integrating LLM-driven reasoning, the system is capable of understanding dynamic Document Object Models (DOMs), interpreting form fields, and navigating application flows—allowing it to operate effectively even in complex or unfamiliar environments. The system’s modular architecture also supports seamless extension to future exploit categories like CSRF, LFI/RFI, authentication bypass, and command execution.

RAIDER is particularly valuable for cybersecurity researchers, small and medium-sized organizations, and educational institutions that lack access to expensive manual red-team assessments. With its integrated GUI dashboard and human-in-the-loop safety controls, RAIDER provides an accessible, affordable, and flexible platform for performing high- quality security audits. Additionally, its use of a mock-target environment enables safe experimentation, training,

and controlled evaluation without risking real-world infrastructure. By bridging the gap between traditional scanners and expert human penetration testers, RAIDER establishes a new standard for intelligent, automated offensive security.

1.5 Organization of Report

The rest of this report is divided into structured chapters as outlined below.

Chapter 2 presents the literature survey, reviewing existing tools, research work, and related systems that form the foundation of autonomous red-teaming and vulnerability exploration. Chapter 3 specifies the system requirements for RAIDER, including both software and hardware dependencies necessary for implementing the framework.

Chapter 4 details the design of the proposed system, covering architectural diagrams, module interactions, the Blackboard communication model, the reinforcement learning workflow, and data flow design.

Chapter 5 explains the methodology and execution plan adopted during the development of RAIDER, along with algorithmic procedures and workflow representations.

Chapter 6 focuses on the implementation, describing the coding structure, modules developed integration of agents, use of LLM APIs, and the functioning of the graphical dashboard.

Finally, Chapter 7 discusses the applications, limitations, and potential future enhancements of RAIDER, including expansions in exploit capability, improvements in agent coordination, and opportunities for broader deployment in real-world cybersecurity environments.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

A Literature Survey is an organized review of existing scholarly work, tools frameworks, and methodologies related to a particular domain. It involve systematically examining books, research papers, academic articles, industry reports, and existing technologies to understand the current state of the art, identify gaps, and position the proposed system in a broader research context. For the RAIDER project, the literature survey helps establish a strong foundational understanding of autonomous penetration testing, reinforcement learning in cybersecurity, multi-agent architectures, exploit generation, and the use of large language models (LLMs) for contextual decision-making. The primary purpose of this literature survey is to gain insights into current research trends and technological advancements in automated security testing. It enables us to analyze how existing penetration testing tools operate, what limitations they face, and how novel AI techniques can address these gaps. Our research spans multiple domains— traditional penetration testing utilities, AI-driven offensive security systems, reinforcement learning frameworks for cyber operations, and emerging work on LLM-based vulnerability analysis.

During the review process, we identified several critical shortcomings in existing automated tools, particularly their reliance on static payload lists, lack of contextual understanding of web applications, and inability to construct intelligent multi-step attack paths. Traditional scanners such as SQLMap, Nikto, and BurpSuite extensions are powerful but heavily signature-driven, which limits their adaptability. Meanwhile, earlier research on reinforcement learning for cybersecurity often focuses on simulated environments rather than real-world, dynamic web interfaces.

Through the study of advanced multi-agent systems and Blackboard architectures, we observed how distributed autonomous agents can collaborate effectively toward a shared objective. Additionally, literature on LLM-driven security analysis demonstrates the growing capability of models like GPT and Gemini to interpret DOM structures, identify weak input points, and generate reasoning-based exploit hypotheses—insights that directly inspire RAIDER’s context-aware exploit research design.

Overall, the literature survey provides a comprehensive understanding of the technological landscape, highlighting both the progress in autonomous cyber operations and the persistent challenges that remain. These insights guide the development of RAIDER, positioning it as

system that unifies reinforcement learning, multi-agent coordination, and LLM-powered security analysis to transcend the limitations of existing penetration testing methodologies.

2.2 Summary of Literature Survey

1. Liang Chen et al. – Deep Reinforcement Learning for Intelligent Penetration Testing Path Design

This paper is foundational for understanding how reinforcement learning can optimize attack-path selection in penetration testing environments. The authors propose the MDDQN algorithm, which integrates attack graphs with Double Deep Q-Networks to improve convergence speed and accuracy in identifying optimal exploit sequences. Their work demonstrates that incorporating structured prior knowledge into RL significantly enhances performance in high-dimensional cyber environments. However, the approach is limited by its dependency on pre-generated attack graphs and its inability to autonomously gather reconnaissance data—highlighting the need for systems like RAIDER that conduct real-time environment exploration and dynamic exploit research.

2. Frank R. Brown & T. Nguyen – Modelling Penetration Testing with Reinforcement Learning Using Capture-the-Flag

This study explores the feasibility of using model-free reinforcement learning for simplified penetration testing tasks inspired by Capture-the-Flag (CTF) scenarios. By modeling tasks such as port scanning and privilege escalation as RL challenges, the paper provides early evidence that Q-learning can autonomously discover effective attack strategies. The research also experiments with imitation learning and state aggregation to reduce training time. Despite its contributions, the study is constrained by highly abstract environments and tabular RL methods that do not scale to real-world conditions. These limitations directly motivate RAIDER’s use of more advanced multi-agent and LLM-assisted techniques.

3. S. Vijay, N. Gupta & P. Mehta – Knowledge-Informed Auto-Penetration Testing Based on RL with Reward Machines

The authors introduce the DRLRM-PT framework, which advances automated penetration testing by embedding expert knowledge into the RL reward structure through Reward Machines. Their approach decomposes penetration testing into structured subtasks, improving interpretability and training efficiency. The DQRM agent in their evaluation demonstrates higher performance and requires fewer steps than standard DQN agents. The paper, however, focuses primarily on lateral movement and does not extend its framework to full attack chains. The heavy reliance on curated domain rules underscores the need for adaptive systems like RAIDER that combine autonomous exploration with reasoning capabilities from large

language models.

4. A. Razak & K. Zhou – Hierarchical Reinforcement Learning for Automated Penetration Testing of Large Networks

This work addresses scalability issues inherent in flat RL approaches by proposing a hierarchical reinforcement learning framework (IAPTF) that breaks complex networks into smaller “security clusters.” Each cluster is treated as a low-level POMDP, while a high-level controller coordinates global decisions. Their results show significant improvements in execution time and effectiveness compared to traditional RL. However, the framework lacks integration with real web environments and does not incorporate LLM-driven reasoning, limiting its ability to navigate dynamic DOM structures—an area where RAIDER provides substantial advancement.

5. Corentin Dumont et al. – Deep Reinforcement Learning for Penetration Testing (arXiv, 2020)

This paper introduces a DRL-based approach for automated penetration testing using attack graphs. It provides foundational insights into how RL agents can explore and exploit networks by learning optimal attack paths. The work aligns with our project’s goal of enabling autonomous vulnerability discovery through intelligent, reward-driven actions.

6. John H. Hammar & Pontus Johnson – Reinforcement Learning-Based Penetration Testing with Attack Graph Reduction (IEEE TDSC, 2018)

This study demonstrates how RL can improve penetration testing efficiency by operating over reduced attack graphs. It validates the feasibility of RL for large-scale, complex networks and influences our use of structured state-action representations to handle multi-step attack planning.

7. Wenqi You et al. – AutoPentest: Network Penetration Testing Using Deep Reinforcement Learning (Computers & Security, 2022)

This work presents an end-to-end automated pentesting framework that uses DRL to learn exploitation strategies. It supports our project by showing how multiple exploitation steps can be sequenced autonomously, reinforcing the need for multi-agent coordination in realistic attack scenarios.

8. Haoran Zhang et al. – Hierarchical Reinforcement Learning for Scalable Penetration Testing (IEEE S&P Workshops, 2021)

The authors propose a hierarchical RL architecture to tackle scalability issues in large networks. Their multi-level POMDP formulation directly informs our project’s strategy of assigning specialized roles to different agents, improving task decomposition and coordinated attack execution.

9.Zhenpeng Chen et al. – Automated Web Vulnerability Exploitation Using Large Language Models (arXiv, 2023)

This paper examines the ability of LLMs to generate and adapt web exploit payloads such as SQLi and XSS. It demonstrates LLM-driven reasoning in offensive security tasks, supporting our integration of LLM-based agents for reconnaissance, payload crafting, and vulnerability interpretation.

10.Samuel C. Bell & Robert K. Min – LLM-Pentest:

Evaluating Large Language Models for Offensive Security Tasks (BlackHat / arXiv, 2023)

2.3 Drawbacks of Existing System

Existing penetration testing tools and automated scanners suffer from several limitations that reduce their effectiveness in modern, dynamic web environments. These drawbacks highlight the need for an intelligent, autonomous system like RAIDER.

1. Reliance on Static Payloads

- * Most tools use predefined payload lists for SQLi, XSS, and other attacks, making them ineffective against applications requiring contextual or dynamic reasoning.

2. Lack of Adaptive Decision-Making

- * Traditional scanners do not learn from previous actions. They repeat the same tests even after failures and cannot optimize attack sequences based on outcomes.

3. No Understanding of Webpage Context

- * Existing tools struggle with dynamic DOMs, JavaScript-generated content, hidden fields, or multi-step forms.
- * They cannot interpret the structure of a page the way LLM-based systems can.

4. Limited Multi-Step Attack Capability

- * Most automated tools perform single-step vulnerability checks and cannot chain attacks (e.g., login bypass → session hijack → XSS escalation).

5. Absence of Autonomous Reconnaissance

- * Conventional tools require manual configuration for scanning. They cannot independently choose targets, explore pages, or decide which paths to follow next.

6. High False Positives and Negatives

- * Due to rule-based detection and lack of reasoning, many scanners misidentify vulnerabilities or miss subtle exploit opportunities.

2.4 Problem Statement

High-quality penetration testing today is still a highly specialized, manual, and resource-

intensive process. Organizations depend heavily on expert red-team professionals who possess deep technical knowledge, creativity, and the ability to plan multi-step attack sequences. However, such expertise is limited, expensive to hire, and cannot be scaled to meet the growing security needs of modern, rapidly evolving web applications. Existing automated scanners offer only superficial vulnerability detection and lack the reasoning, adaptability, and collaborative intelligence required to uncover complex, chained exploits. They operate in isolation, rely on static rule sets, and fail to replicate the strategic thinking of human attackers. As a result, many organizations—especially small and medium enterprises—are unable to perform frequent, in-depth security assessments due to high cost, limited availability of experts, and the inefficiency of current automated tools.

There is a critical need for a fully autonomous system that can replicate human-like red-team dynamics by combining reconnaissance, decision-making, multi-step planning, and exploitation. Such a system must be capable of discovering hidden vulnerabilities, understanding contextual information, coordinating between specialized agents, and executing sophisticated attack chains— tasks far beyond the capability of traditional scanners.

Addressing this gap will enable scalable, affordable, and continuous penetration testing, ensuring that advanced security audits become accessible to all sectors rather than a privileged few. The RAIDER project aims to fill this gap by building an intelligent multi-agent system that autonomously plans and performs end-to-end penetration testing with human-level reasoning and coordination.

2.5 Proposed Solution

1. Multi-Agent Architecture & Role Specialization:

Existing penetration testing tools operate as monolithic scanners, lacking the collaborative intelligence of real red teams. RAIDER addresses this limitation by implementing a multi-agent architecture, where each agent is designed for a specific security function—Reconnaissance, Exploitation, Coordination, Logging, and Monitoring. The agents communicate through a shared Blackboard system, enabling asynchronous information exchange (e.g., discovered endpoints, open ports, failed payloads). This design allows each agent to act independently while contributing to a unified attack strategy. The modular approach ensures scalability, easier debugging, and future extensibility (e.g., privilege escalation agent, lateral movement agent).

2. LLM-Based Context Awareness & Payload Generation: Traditional exploit tools rely on static payload lists, making them ineffective against dynamic web applications. RAIDER integrates a Large Language Model (Gemini) to provide deep contextual reasoning during exploitation.

The Exploit Agent uses LLM intelligence to:

Analyze DOM structures via Selenium to understand form fields, input types, and hidden controls. Generate context-aware SQL Injection and XSS payloads rather than using static dictionaries. Determine the best navigation paths (e.g., specific buttons or forms to target). Adapt payloads based on page responses, enabling iterative refinement. Using LLMs transforms the system from a brute-force scanner into an intelligent exploit researcher capable of discovering novel attack vectors.

3. Reinforcement Learning for Autonomous Decision-Making:

To mimic real red teams, RAIDER integrates Q-Learning–based Reinforcement Learning within the Coordination Core. Instead of executing predefined sequences, the system learns optimal attack strategies through repeated interaction with targets and mock environments.

The RL agent:

Maintains a Q-table representing actions such as scanning, exploiting, switching payload types, or navigating DOM paths. Uses rewards shaped around successful exploits, page responses, privilege gains, or discovery of new vulnerable endpoints.

Continuously improves policy decisions for multi-step attacks (e.g., recon → exploit → escalate). This enables RAIDER to autonomously adapt to new environments and minimize redundant actions, making it increasingly efficient over time

4. Autonomous Reconnaissance & Environment Understanding:

RAIDER’s Recon Agent replaces manual enumeration by combining classical tools with AI reasoning. Key features include:

Automated scanning using Nmap to detect ports, services, OS details, and exposed interfaces. Intelligent parsing of scan outputs to prioritize high-value attack surfaces. Logging findings into the Blackboard so the Exploit Agent can make informed decisions. Support for both live targets and simulated environments (mock_target.py). This automated reconnaissance ensures that the Exploit Agent operates with accurate, real-time intelligence.

5. Human-in-the-Loop Safety, Control & Transparency:

Full autonomy in security systems demands strong safety measures. RAIDER includes a dedicated. Interactive GUI Dashboard for real-time human oversight. The dashboard allows operators to: Pause, resume, or terminate attacks at any time. Monitor ongoing actions, agent decisions, and reconnaissance findings. View exploit attempts and system reasoning logs. Approve or override

high risk actions before execution. This ensures that even though the system is autonomous, operators retain full situational awareness and governance.

6. Robust Logging, Reporting & Auditability:

Accurate documentation is essential for professional penetration testing. RAIDER includes a Logging and Reporting Module that:

Stores each agent's actions, payloads attempted, responses received, and final outcomes. Track Q-learning improvements across episodes to visualize learning progression. Generates structured penetration testing reports for organizations. Supports forensic analysis by recording screenshot of DOM states during exploitation. This ensures transparency, reproducibility, and compliance.

CHAPTER 3

REQUIREMENT ENGINEERING

3.1 Hardware and Software Tools Used

3.1.1 Hardware Tools Used

Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5	Intel Core i7 / AMD Ryzen 7 or higher
Memory (RAM)	8 GB or higher	16 GB or higher
Storage	256 GB SSD	512 GB SSD
Graphics	Integrated graphics (sufficient for Selenium automaion)	Dedicated GPU preferred (for RL model scaling)
Internet Connectivity	Stable internet connection for LLM (Gemini/OpenAI)	High-speed internet for faster model inference & package installation

Table 3.1 Hardware Requirements

3.1.2 Software Tools Used

Category	Tool / Requirements
Operating System	Windows 10/11, Ubuntu 20.04+ / Kali Linux (recommended for penetration testing)
Development Environment	Python 3.10+, Virtual Environment / Conda
Core Libraries & Frameworks	Selenium, Requests / urllib, Nmap & python-nmap, OpenAI / Gemini API, Q-Learning & RL Libraries (NumPy, Pandas, Matplotlib), Flask, BeautifulSoup (optional), Joblib / pickle
GUI Framework	Tkinter / CustomTkinter / Streamlit (for dashboard & controls)
Browsers & Drivers	Google Chrome + Compatible ChromeDriver
Penetration Testing Tools	Nmap, Burp Suite (<i>optional</i>), Metasploit Framework (<i>optional</i>)
Version Control	Git / GitHub

Table 3.2 Software Requirements

3.2 Software Requirements Specifications

3.2.1 Functional Requirements

1. Target Input & Configuration Module

The system shall allow the user to input the target URL/IP address. The system shall allow the user to configure scanning options (full scan, web-only scan, recon-only, exploit-only). The system shall validate the target format before initiating tests.

2. Coordination Core (RL Strategy Agent)

The RL agent shall read the current attack state from the Blackboard. The RL agent shall select an optimal action (DEPLOY_RECON, DEPLOY_SQLI, DEPLOY_XSS) using Q-learning. The RL agent shall update the Q-table after each step based on reward feedback. The RL agent shall send commands to Recon Agent or Exploit Agent based on its decision.

3. Recon Agent (Automated Discovery Module)

The Recon Agent shall execute Nmap scans (port scan, service scan, OS scan). The system shall collect and parse Nmap results (open ports, protocols, service versions). The Recon Agent shall write scan results to the Blackboard. The Recon Agent shall notify the RL agent after recon is complete.

4. Exploit Agent (LLM-based Tactical Exploitation Module)

The Exploit Agent shall receive exploitation commands from the RL agent. The system shall interact with the LLM API (Gemini) to generate attack payloads (SQLi, XSS). The Exploit Agent shall use Selenium WebDriver to load web pages and analyze the DOM. The Exploit Agent shall inject generated payloads into input fields automatically. The system shall analyze server responses and detect unusual behavior (alerts, database errors, script execution).FR4.6: The Exploit Agent shall store exploitation results, flags, and vulnerability findings in the Blackboard. The Exploit Agent shall return success/failure states to the RL Agent for reward calculation.

5. Blackboard (Shared Memory Module)

The system shall maintain a shared data structure accessible by all agents. The Blackboard shall store recon scan results exploit results target metadata current RL state

The Blackboard shall allow agents to read/write concurrently. The system shall prevent data corruption through controlled access.

6. External Tool Integration

The system shall execute Nmap commands and parse their output. The system shall interact with Selenium WebDriver for browser automation. The system shall communicate with the LLM API for payload reasoning. The system shall handle API failures and retry requests.

7. Attack Path Execution

The system shall autonomously chain steps in the sequence:

Recon → Exploit → Evaluate → Escalate

The system shall detect multi-step vulnerability chains (e.g., weak login → injection point → JavaScript execution). The system shall repeat attacks until the RL agent converges or max attempts are reached.

8. Reporting & Output Module

The system shall generate a structured vulnerability report summarizing:

Open ports discovered vulnerabilities payloads used success status exploitation screenshots (optional)

The system shall export findings in PDF/HTML format. The system shall maintain logs for each attack step.

9. GUI Dashboard (User Interface Module)

The system shall display all real-time events:

scan progress exploit attempts RL decisions The dashboard shall provide controls to start, pause, or stop the attack. The dashboard shall show final results and recommended mitigation steps.

10. Security & Access Control

The system shall restrict usage to authorized users. The system shall ensure that only user-defined targets can be tested. The system shall not store sensitive data without user permission.

3.2.2 Non-Functional Requirements

1. Scalability

The system must be capable of handling increasing levels of complexity without performance degradation. RAIDER should support multiple agents operating concurrently (Recon, Exploit, RL Coordinator). As target environments grow in size (e.g., more endpoints or services), the system must maintain consistent performance. The architecture should allow integration of additional agents in the future, such as privilege escalation or lateral movement agents.

2. Modularity

Each component of the system should be designed as an independent module. The Recon Agent, Exploit Agent, Coordination Core, and GUI must function autonomously while communicating through the Blackboard. Updating or replacing one module (e.g., adding a new exploit type) must not disrupt others. This modular design ensures easier debugging, testing, and extension of the system.

3. Security

Given that RAIDER performs offensive security tasks, the system must enforce strict control and safety standards. All exploit execution must occur in isolated or simulated environments (e.g., `mock_target.py` or sandbox VMs). The GUI must allow pausing/terminating attacks to prevent unintended harm. Sensitive outputs (payloads, logs, credentials) must be stored securely. Prevent

misuse by ensuring agents cannot operate on unauthorized targets.

4. Reliability

RAIDER should maintain stable performance across multiple testing sessions. Agents must handle unexpected errors (e.g., failed requests, DOM changes, driver crashes) without system failure. The RL model must save and load Q-tables reliably for consistent learning behaviour. Logging must continue even if individual agents encounter errors.

5. Maintainability

The system should be structured to support long-term updates and improvements. Clean, well-documented code makes it easier for developers to modify or add features. Modules must follow clear naming conventions and organized directories. Error logs and debugging information should be accessible for troubleshooting.

CHAPTER 4

DETAILED DESIGN

4.1 System Architecture

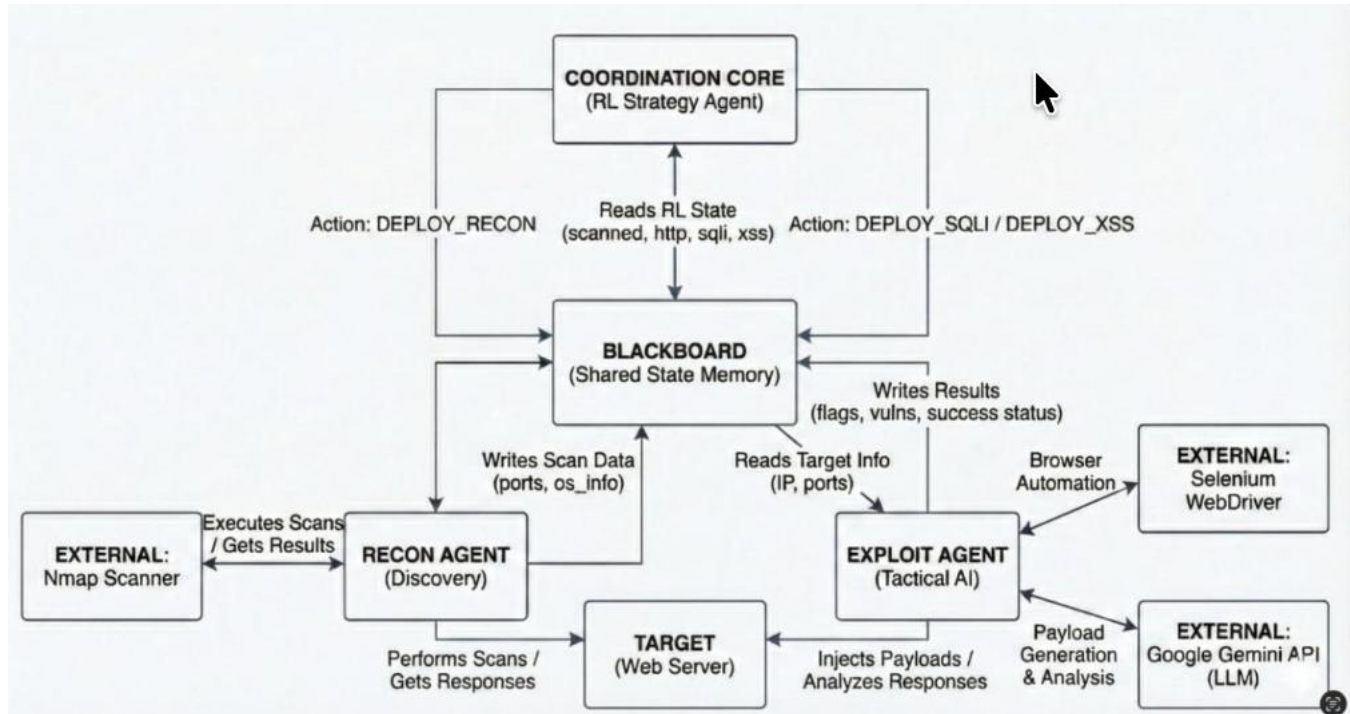


Figure 4.1: RAIDER Backbone

The system architecture of RAIDER – Autonomous Red Teaming System follows a modular, multi-agent design where each agent performs a specialized role in the penetration-testing workflow. The architecture enables autonomous reconnaissance, vulnerability analysis, and exploitation through coordinated interaction between the Reinforcement Learning (RL) strategy agent, the Recon Agent, the Exploit Agent, and external tools such as Nmap, Selenium WebDriver, and Google Gemini LLM.

The system is built around three core architectural pillars:

Coordination Core (RL Strategy Agent)

Blackboard (Shared State Memory)

Specialized Agents (Recon Agent and Exploit Agent)

All modules communicate through the centralized Blackboard, enabling seamless data flow, shared knowledge, and collaborative decision-making.

4.1.1 Coordination Core (RL Strategy Agent)

The Coordination Core serves as the brain of the system, responsible for planning multi-step attack paths. It uses a Reinforcement Learning (RL) model to evaluate the current security state of the target and decide the next appropriate action. Responsibilities:

Reads the current RL state from the Blackboard (scan results, HTTP responses, SQLi/XSS status, etc.)

Selects actions such as:

DEPLOY_RECON

DEPLOY_SQLI

DEPLOY_XSS

Sends instructions to the agents based on learned attack strategies. Continuously updates its policy by analyzing success/failure of previous attempts. The Coordination Core ensures the attack sequence is adaptive, optimized, and autonomous—similar to the operation of a human red-team strategist.

4.1.2 Blackboard (Shared State Memory)

The Blackboard functions as a centralized intelligence hub where all agents read and write information.

Key Functions:

- Stores scan details (open ports, OS info, technology stack)
- Stores target information (IP address, endpoints)
- Stores exploitation results (validation flags, vulnerabilities found, success metrics)
- Facilitates communication between Recon Agent, Exploit Agent, and RL Agent
- Maintains synchronized system state for decision-making

This architecture ensures loose coupling between agents while enabling highly coordinated, multi-step attack chains.

4.1.3 Recon Agent (Discovery Module)

The Recon Agent is the system's discovery and information-gathering module. It is responsible for scanning the target environment and identifying potential attack surfaces.

1. Functions:

- Executes external scanning tools such as Nmap

2. Performs:

- Port scanning

- Service fingerprinting
- OS detection
- Writes scan results and environment information to the Blackboard
- Passes actionable intelligence to the Exploit Agent
- The Recon Agent provides the foundational data necessary for effective exploitation planning.

4.1.4 Exploit Agent (Tactical AI Module)

The Exploit Agent serves as the tactical attack executor, focusing on payload generation, injection, and exploitation.

Key Capabilities:

- Utilizes Google Gemini LLM for intelligent payload generation (SQLi, XSS, parameter tampering)
- Performs browser automation using Selenium WebDriver
- Reads target info (ports, endpoints, services) from the Blackboard
- Analyzes server responses to determine exploit effectiveness
- Writes results (flags, vulnerabilities, success metrics) back to the Blackboard
- This agent represents the human-like reasoning and creativity of the autonomous red team.

4.1.5 Target Module (Web Server / Application Under Test)

The target environment represents:

- A simulated or real web application
- Configured with endpoints vulnerable to SQL Injection, XSS, authentication flaws, etc.
- Both Recon and Exploit Agents interact with the target:
- Recon performs scans
- Exploit injects payloads and analyzes responses
- This modular setup supports iterative testing and reinforcement learning.

4.1.6 External Tools Integration

- Nmap Scanner :provides high-fidelity reconnaissance integrated with Recon Agent
- Selenium WebDriver:automates browser interactions enables DOM analysis, form submission, and dynamic payload injection
- Google Gemini API (LLM):generates intelligent and context-aware exploit payloads interprets web responses and helps refine attack strategies

These external tools greatly enhance the system's accuracy and autonomy.

4.1.7 Overall Workflow

- RL Strategy Agent decides the next action (e.g., Deploy Recon).
- Recon Agent executes Nmap scans and writes results to the Blackboard.
- RL Agent reads updated state and decides next step (e.g., SQL Injection test).

- Exploit Agent generates payloads using Gemini, injects them into the target, and analyzes results.
- Exploitation results are stored in Blackboard.
- RL Agent learns from success/failure and refines future strategies.

This cycle continues until a valid vulnerability chain is discovered, or the attack plan is exhausted.

4.2 Data Flow Diagram

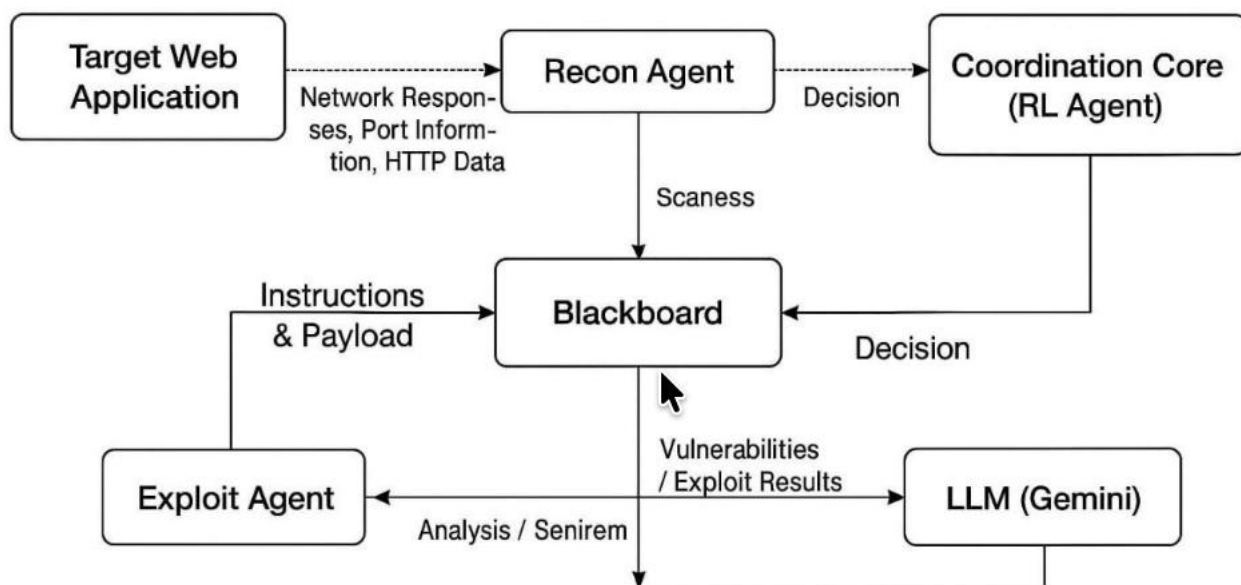


Fig. 4.2 Workflow for Autonomous Red Teaming & Vulnerability Exploitation

The Data Flow Diagram (DFD) illustrates the complete workflow of the **RAIDER** Autonomous Red Teaming System, showing how information flows between major components—including agents, external tools, and the target application.

The process begins with the Target Web Application, which provides raw network responses, open-port data, and HTTP metadata. This information is passed to the Recon Agent, which performs scanning tasks using external tools such as Nmap. The Recon Agent extracts port information, service banners, OS fingerprints, and preliminary vulnerability indicators.

All reconnaissance results are written to the Blackboard, a centralized shared-memory system that stores intermediate data accessible to all agents. The Coordination Core (RL Strategy Agent) reads this data from the Blackboard and determines the next step—either redeploying reconnaissance or initiating exploitation. Its decision is then written back to the Blackboard.

Based on these decisions, the Exploit Agent retrieves instructions and payloads from the Blackboard. It performs attack execution using browser automation and interacts with the LLM (Gemini API) to

generate context-aware payloads, analyze responses, and identify exploit patterns such as SQL injection and XSS vectors. The LLM's analysis and exploit results are fed back into the Exploit Agent and subsequently written into the Blackboard as verified vulnerabilities or exploitation outcomes. This updated knowledge base enables the Coordination Core to refine its strategy, creating a continuous closed-loop cycle.

4.3 Algorithm Design

1. Reconnaissance Algorithm (Nmap-Based Scanning) Algorithm ReconScan(target):

Input: Target IP / URL

Output: List of open ports, services, and endpoint hints

1. Initialize Nmap scanner

1. Execute scan using command:

```
nmap -sV -T4 <target>
```

2. Parse scan output:

For each port in results:

extract:

- port number
- service name
- version banner

3. Identify possible web endpoints:

if service contains ("http", "https"):

add endpoint = "http://<target>:<port>"

4. Construct structured scan_data:

```
scan_data = {ports, services, endpoints}
```

5. Write scan_data to BLACKBOARD

6. Return scan_data

2. DOM Extraction Algorithm (Selenium-Based) Algorithm ExtractDOM(url):

Input: Target URL

Output: DOM tree elements, form fields, clickable components

1. Launch Selenium WebDriver (Headless Mode)
2. Navigate to URL
3. Wait for page to render (implicit wait)

4. Extract DOM structure:

- document root HTML
- list of <input> fields
- list of <form> elements
- list of clickable elements (<button>, <a>, <script> triggers)

5. Create simplified DOM skeleton:

```
skeleton = {
  inputs: [...],
  buttons: [...],
  forms: [...],
  raw_html: ...
}
```

6. Write skeleton to BLACKBOARD

7. Return skeleton

3. **PayloadGenerationAlgorithm**(LLM-Assisted)

1. Algorithm GeneratePayload(dom_skeleton, vuln_type):

2. Input: DOM skeleton, vulnerability type (SQLi / XSS)

3. Output: Context-aware payload list # Try models in priority order

[08/12/25, 3:23:07 PM] Bibi Mariam:

1. Construct LLM

```
prompt: prompt =
  "Given this DOM structure: {dom_skeleton},
  generate payloads for: {vuln_type}.
  Return only a list of payloads."
```

2. Send prompt to Google Gemini LLM API

3. If API returns payloads:

```
payload_list = response.payloads
```

4. **Else:**

Use fallback payloads:

```
if vuln_type == SQLi:
  payload_list = ["' OR '1'='1", "' UNION SELECT NULL
  --"]
if vuln_type == XSS:
  payload_list = ["<script>alert(1)</script>", "\" onmouseover=alert(1)"]
```

1. For each payload in payload_list:

- a. Load page using Selenium
- b. Identify injection points from DOM skeleton
- c. Insert payload into input/textarea fields
- d. Submit form or trigger associated button
- e. Capture response:
 - page source
 - error messages
 - DOM changes
 - JS alerts (for XSS)

2. Evaluate success:

If SQL error or DB trace found:

return {success = True, type = "SQLi", evidence}

If injected script executed:

return {success = True, type = "XSS", evidence}

3. If no payload succeeded:

return {success = False}

5. Blackboard Communication Algorithm

Algorithm BlackboardUpdate(agent_id,

data): Input: Agent name, data dictionary

Output: Updated shared memory

1. Read current BLACKBOARD state
2. Append/update entry:

BLACKBOARD[agent_id] = data
3. Save synchronized state
4. Notify subscribed agents (if message bus enabled)
5. Return updated BLACKBOARD

6. RL Coordination Algorithm (Q-Learning

Policy) Algorithm RL_Coordinator(state):

Input: RL State (recon_done, endpoints_found, sqli_success, xss_success)

Output: Action (DEPLOY_RECON / DEPLOY_SQLI / DEPLOY_XSS / STOP)

1. Define actions $A = \{\text{DEPLOY_RECON}, \text{DEPLOY_SQLI}, \text{DEPLOY_XSS}, \text{STOP}\}$
2. If episode just started:

return DEPLOY_RECON
3. Choose action using epsilon-greedy

policy: with probability ϵ :

action =

random(A) else:

action = $\text{argmax}(Q[\text{state}])$

4. Execute selected action:

if action == DEPLOY_RECON:

trigger Recon Agent

if action == DEPLOY_SQLI:

trigger Exploit Agent (SQLi mode)

if action == DEPLOY_XSS:

trigger Exploit Agent (XSS mode)

5. Receive reward:

+100 for confirmed exploit

+10 for new endpoints

discovered -5 for invalid attempt

-1 for repeated or idle actions

-50 for forbidden/dangerous behavior

6. Update Q-table:

7. Save Q-table

8. Return action

7. Mission Logging & Reporting

Algorithm Algorithm GenerateReport(logs):

Input: Mission logs (scan data, payloads, results, RL decisions)

Output: Structured JSON + PDF report

1. Collect all logs from BLACKBOARD:

- Recon results
- DOM data
- Payload generation details
- Exploit attempts & evidence
- RL decisions

2. Structure mission_report = {

```

target_info,
recon_summary,
exploit_summary,
success_flags,
recommendations
}
3. Export mission_report.json
4. Convert JSON → PDF using template engine
5. Save pdf_report
6. Return pdf_report

```

4.4 Sequence Diagram

A sequence diagram illustrates the time-ordered interaction between the major components of the RAIDER autonomous red-teaming system. It shows how the User, Coordination Core, Blackboard, Recon Agent, Exploit Agent, and Target System exchange messages to perform reconnaissance, analysis, payload generation, and exploitation.

This diagram captures the autonomous workflow that begins with mission deployment and ends with exploit execution and feedback integration into the RL-based Coordination Core.

1. Mission Initialization: The User initiates the attack workflow by triggering an automated red-team mission. The Coordination Core (RL Strategy Agent) receives the request and formulates the first action.

2. Reconnaissance Phase: Coordination Core → Blackboard: DEPLOY_RECON

RL agent instructs the system to begin reconnaissance. Blackboard → Recon

Agent: SCAN_TARGET. Blackboard sends target details (IP, ports) to the Recon

Agent. Recon Agent → Target: Performs Nmap scans.

Recon Agent → Blackboard: Writes scan results (open ports, OS info, HTTP services).

Blackboard → Coordination Core: RL state update with recon results.

3. Exploitation Decision: Based on recon data, the Coordination Core determines whether to deploy SQL Injection testing, perform DOM analysis for XSS. Trigger browser-based exploration. Coordination Core → Blackboard: DEPLOY_SQLI / DEPLOY_XSS

4. Payload Generation & Analysis: Blackboard → Exploit Agent: Sends selected exploit type + target context.

Exploit Agent → Gemini LLM API: Requests payload generation (SQLi, XSS, or fuzz payloads).

5. Payload Injection: Blackboard → Exploit Agent: INJECT_PAYLOAD instruction after RL

approval. Exploit Agent → Target: Injects payload (SQLi query, XSS script, form bypass attempt).

6. Response Evaluation: Exploit Agent → Blackboard: Writes vulnerability flags, response characteristics, success metrics.

Blackboard → Coordination Core: Provides updated RL state.

Coordination Core: Evaluates reward, adjusts strategy, determines next step (continue / escalate / stop)

7. Mission Output:Blackboard → User: Final results include Recon summary, detected vulnerabilities, proof-of-concept payloads, screenshots/HTML content if XSS succeeds
System concludes with an autonomous cycle of recon → exploitation → evaluation.

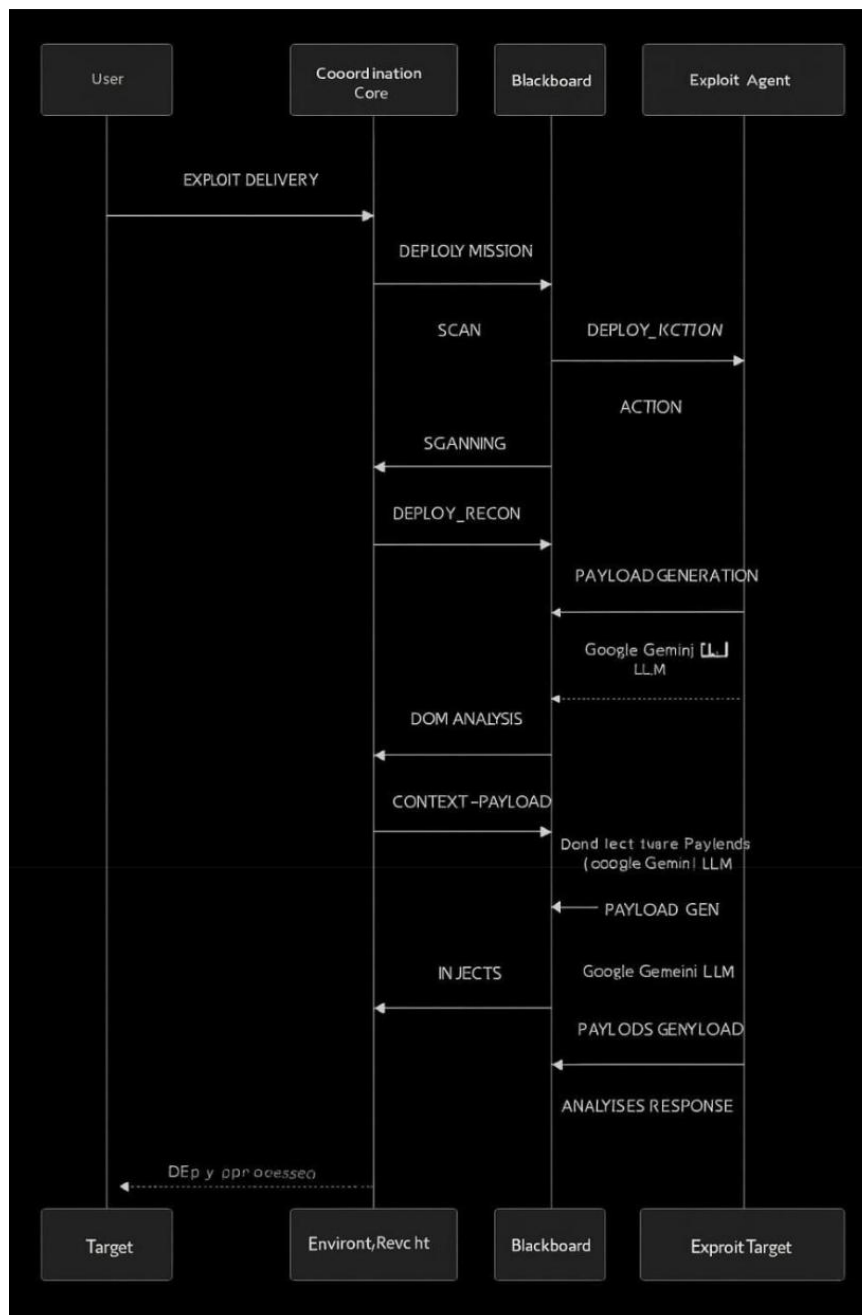


Fig. 4.3 Sequence diagram of the Red Teaming Backbone

CHAPTER 5

IMPLEMENTATION

5.1 Overview of Implementation

This chapter describes the concrete implementation of **RAIDER**, an autonomous red-teaming system that integrates multi-agent reinforcement learning, LLM-assisted exploit research, and classical reconnaissance tools to perform context-aware penetration testing. Given a target IP/URL, RAIDER performs automated reconnaissance using Nmap, analyzes dynamic web pages using Selenium, generates context-aware payloads with an LLM (Google Gemini), and orchestrates agents via a Reinforcement Learning (Q-learning) Coordination Core. The system supports both real targets and a safe simulated environment (`mock_target.py`) for training and validation. A Blackboard shared memory enables agents to publish findings and coordinate actions, while a GUI dashboard provides Human-in-the-Loop controls (pause/resume/terminate), live logs, and reporting.

1. Reconnaissance (Recon Agent)

The Recon Agent performs discovery using Nmap and auxiliary HTTP probes. Scans are executed programmatically (`python-nmap`) and parsed to extract open ports, services, version banners, and probable web endpoints. Extracted data (IP, ports, service versions, endpoints) are written to the Blackboard for downstream processing.

2. Target Profiling & Preprocessing

Scan outputs and retrieved pages are normalized: service banners are standardized, endpoints are canonicalized, and candidate URLs are queued. For web targets, an initial Selenium pass obtains the DOM skeleton and identifies interactive elements (forms, buttons, inputs, hidden fields). The extracted DOM and metadata are saved to the Blackboard for LLM analysis.

3. Exploit Research & Payload Generation (Exploit Agent)

The Exploit Agent reads DOM skeletons and prompt-engineers the LLM to suggest contextual payloads (SQLi, XSS). Using Selenium, it injects payloads, monitors responses (error messages, DOM changes, alert execution), and evaluates success heuristics. Each attempt (payload, target field, timestamp, result) is logged to the Blackboard.

4. Coordination Core (RL Strategy Agent)

The Coordination Core implements Q-learning to choose high-level actions (DEPLOY_RECON, DEPLOY_SQLI, DEPLOY_XSS, NAVIGATE, STOP). States are derived from Blackboard summaries (scans completed, endpoints discovered, success flags). Rewards are assigned for meaningful progress (new endpoints, confirmed exploit). The Q-table is persisted and visualized via `inspect_brain.py`.

5. Human-in-the-Loop Dashboard & Reporting

A graphical dashboard provides real-time mission telemetry, agent logs, screenshots, and Q-value summaries. Operators can start/stop missions, pause/resume agents, and approve high-risk actions. Completed missions produce structured reports (JSON + auto-generated PDF) containing scan results, payloads attempted, exploitation evidence (screenshots, response excerpts), and recommendations.

5.2 Frontend Development

1. User Interface & Inputs: The GUI is implemented in Streamlit (or an alternative such as PyQt/CustomTkinter if preferred). The interface accepts a target URL/IP, scan depth (quick/full), and agent configuration options (enable/disable Exploit Agent, enable HITL). A clean `text_input` and slider provide straightforward input control.

2. Mission Control Buttons: Buttons for Start Mission, Pause, Resume, and Terminate trigger backend workflows through REST endpoints or direct function calls. The Start action enqueues mission parameters, while Pause/Resume signal the Coordination Core to suspend or continue policy execution.

3. Live Logs & Visual Feedback: The main panel shows live logs streamed from the Blackboard (recon events, exploit attempts, RL decisions). Loading spinners and status badges indicate current agent states. Screenshots captured by Selenium are displayed inline for each exploit attempt.

4. Attack Timeline & Visualization: A timeline component visualizes the sequence of actions (scan → DOM fetch → payload generation → payload injection → validation). Reinforcement learning metrics (episode, step, reward, Q-value delta) are shown in a compact widget for debugging and demonstration.

5. Downloadable Output & Export: Users can download the mission report (PDF) and a CSV of attempted payloads and results. Exported logs include raw Nmap output, sanitized screenshots, and Q-table snapshots for reproducibility.

6. Safety & Permission Dialogs: Before launching on a live target, the UI requires explicit confirmation with a target authorization checkbox and a short authorization text to ensure legal and ethical usage.

5.3 Backend Development

1. **API Key Management & Secure Configuration:** LLM API keys (Gemini/OpenAI) and other secrets are stored in environment variables or streamlit. secrets (for Streamlit deployment). Access to the backend is protected and keys are never hardcoded. Rate-limits and retry policies are configured to handle transient API errors.

2. **Recon Agent Implementation:** Uses python-nmap to run targeted scans and version detection. Parses output to extract services, versions, and likely endpoints. Writes structured scan results to Blackboard (JSON schema: {ip, port, service, banner, endpoints}). Provides a lightweight HTTP probe for endpoint validation.

3. **DOM Extraction:** Selenium WebDriver captures the page DOM, input elements, and any client-side scripts.

4. **LLM Integration:** Prompts are built with DOM skeletons and context; Gemini returns suggested payload templates and navigation hints.

5. **Injection & Validation:** Payloads are injected; response patterns (SQL errors, JS alerts, reflected payloads) determine success.

6. **Fallbacks:** If LLM fails or network is unavailable, the agent uses a local fallback payload set and rule-based heuristics.

7. **Coordination Core & RL Training:** Q-learning implementation (train_manager.py) defines the state space and action set. Rewards: +100 for confirmed exploit, +10 for new endpoint discovery, -5 for invalid attempts, -50 for destructive or forbidden actions. Q-table persistence uses pickle/joblib; inspect_brain.py visualizes Q-values and policy progression.

8. **Blackboard (Shared State Memory):** Implemented as a thread/process-safe in-memory store (Redis optional for distributed setups) with JSON entries and indexed keys. Supports updates.

9. **Logging, Persistence & Reporting:** All agent actions are appended to an append-only log (structured JSON). Screenshots and raw responses are persisted to disk with references inside the logs. Final report generation aggregates logs, evidence, and recommendations.

10. Mock Environment (mock_target.py): A Flask app simulating vulnerable endpoints (login page with SQLi, comment form with XSS) used for safe training and evaluation. Allows reproducible training runs and unit tests for exploit modules.

11. Testing & Deployment: Unit tests for parsing Nmap output, DOM extraction, and payload evaluation.

5.3 Code

Coordination Core:

```
class CoordinationCore:
    def __init__(self, blackboard):
        self.board = blackboard
        self.q_table = {}
        self.actions = ["DEPLOY_RECON", "DEPLOY_SQLI", "DEPLOY_XSS", "WAIT"]
        self.lr = 0.1
        self.gamma = 0.9
        self.epsilon = 0.9
        self.filename = "mission_control.pkl"
        self.load_brain()

    def get_state_key(self):
        return self.board.get_r1_state()

    def choose_action(self):
        state = self.get_state_key()
```

Blackboard:

```
class Blackboard:
    def __init__(self):
        self.state = {
            "target_ip": "",
            "target_url": "",
            "status": "IDLE",
            "scanned": False,
            "ports": [],
            "os_info": "Unknown",
            "mac_address": "Unknown",
            "topology": [],
            "network_vulns": [],
            "vulnerabilities": [],
            "credentials": [],
            "flag_captured": False,
            "sql_success": False,
            "xss_success": False,
            "activity_log": []
        }
```

Recon Agent:

```
def run(self):
    target = self.board.state["target_ip"]
    print(f"{Fore.BLUE}[Recon] Starting Scan on {target}...{Fore.RESET}")

    if self.nm:
        try:
            arguments = (
                '-p 80,443,3000,5000,8080,8080 '
                '-sV -O -TS --max-retries 1 --script-timeout 10s '
                '--script=http-sql-injection,http-methods,http-title,http-headers'
            )
            try:
                self.nm.scan(target, arguments=arguments)
```

Exploit Agent:

```
class ExploitAgent:
    def __init__(self, blackboard, api_key):
        self.board = blackboard
        genai.configure(api_key=api_key)
        self.model = genai.GenerativeModel('gemini-2.5-flash-preview-09-2025')
        self.history = []

    def get_driver(self):
        options = webdriver.ChromeOptions()
        options.add_argument("--log-level=3")
        # options.add_argument("--headless")
        return webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=options)

    def run(self):
        ports = self.board.state["ports"]
        target_ip = self.board.state["target_ip"]
        stored_url = self.board.state.get("target_url", "")
        http_ports = [p for p, s in ports.items() if 'http' in s]
```

Fig. 5.1 Code Snippets

CHAPTER 6

MAPPING TO SDG

The development of an autonomous AI-driven penetration testing and cybersecurity system directly contributes to several United Nations Sustainable Development Goals. The following mapping highlights how the project aligns with global sustainability objectives:

SDG 9: Industry, Innovation, and Infrastructure:

- **Relevant Because:** Cybersecurity is a backbone of resilient digital infrastructure.
- **Project Contribution:** Enhances resilience of digital services by identifying vulnerabilities early. Promotes innovation through autonomous LLM-based and reinforcement-learning-based red teaming. Supports secure adoption of emerging technologies in enterprises and government systems.

SDG 11: Sustainable Cities and Communities:

- **Relevant Because:** Smart cities rely heavily on secure, interconnected digital infrastructure.
- **Project Contribution:** Protects critical smart-city systems (IoT, public services, traffic systems) from cyber threats. Strengthens community safety by preventing cyberattacks on essential digital platforms.

SDG 16: Peace, Justice, and Strong Institutions

- **Relevant Because:** Cybersecurity directly influences national security, digital trust, and institutional integrity.
- **Project Contribution:** Enhances the ability of institutions to defend against cybercrime. Improves transparency and accountability by automating security testing and reducing human error. Assists organizations in maintaining compliance with cybersecurity policies and standards.

SDG 4: Quality Education (Indirect Contribution)

- **Relevant Because:** Cybersecurity education is essential for a modern workforce.
- **Project Contribution:** Provides a high-quality learning tool for training students in ethical hacking and security automation. Demonstrates practical applications of AI, LLMs, and MARL in real-world cybersecurity.

SDG 8: Decent Work and Economic Growth (Indirect Contribution)

- **Relevant Because:** Cyberattacks cause financial losses and operational downtime.

- **Project Contribution:** Reduces economic risks by providing continuous, low-cost, automated security assessments. Supports safer digital environments for businesses and startups.

SDG 17: Partnerships for the Goals (Supporting Contribution)

- **Relevant Because:** Global cybersecurity requires cooperation and shared innovation.
- **Project Contribution:** The system can be extended to support cross-organizational red-team/blue-team collaboration. Encourages industry–academia partnerships in security research and AI innovation.

CHAPTER 7

RESULTS

7.1 Snapshots

The screenshot displays the RAIDER Command Centre interface. At the top, it shows 'STEP: 3/30', 'REWARD: 100', and 'DEPLOY_SQLI'. The main area is divided into 'MISSION LOGS' and 'INTELLIGENCE' sections.

MISSION LOGS:

```
[!] http-methods: Supported Methods: GET HEAD POST OPTIONS...
[!] http-headers: Server: Apache-Coyote/1.1 Set-Cookie: JSESSIONID=8237E42D3...
[!] http-server-header: Apache-Coyote/1.1...

[Blackboard] INTELLIGENCE UPDATE:
:: Open Ports: [80, 443, 8080]
:: OS Detected: Unknown (Cloud/Firewall Protected)
:: MAC Address: Unknown (Remote Target - Layer 2 Hidden)

--- Mission Step 2 ---
[Commander] Decision: DEPLOY_XSS
[XSS-Agent] Hunting for Reflected XSS on http://demo.testfire.net...
[XSS-Agent] AI Analyzing page structure for vectors...
[XSS-Agent] AI Decision: Stay on current page (A visible search input field is present on the c
urrent page, which is a high-priority target for testing Reflected XSS.)
[XSS-Agent] Locked on Target Page: https://demo.testfire.net/
[XSS-Agent] Found 1 vectors. Ranking priorities...
[XSS-Agent] Attack Order: ['query']

[XSS-Agent] >>> FOCUSING ON INPUT: 'query' (context: main) <<<
[XSS-Agent] Attempt 1/3 on 'query': <script>alert('XSS_SUCCESS')</script>
[XSS-Agent] XSS CONFIRMED! Alert text: XSS_SUCCESS (via Post-Submit Wait)

[Blackboard] CRITICAL: Flag Captured -> FLAG(XSS_PAYLOAD_EXECUTED)
[Blackboard] Vulnerability Logged: Reflected XSS on https://demo.testfire.net/ | Input: query |
Payload: <script>alert('XSS_SUCCESS')</script>

--- Mission Step 3 ---
[Commander] Decision: DEPLOY_SQLI
[Exploit] Targeting: http://demo.testfire.net
```

INTELLIGENCE:

```
[Blackboard] INTELLIGENCE UPDATE:
:: Open Ports: [80, 443, 8080]
:: OS Detected: Unknown (Cloud/Firewall Protected)
:: MAC Address: Unknown (Remote Target - Layer 2 Hidden)

DETAILED PORT ANALYSIS:
-> Port 80: http
[!] http-title | Altoro Mutual
[!] http-methods | Supported Methods: GET HEAD POST OPTIONS
[!] http-headers | Server: Apache-Coyote/1.1 Set-Cookie: JSESSIONID=894EB4875
[!] http-server-header | Apache-Coyote/1.1
-> Port 443: http
[!] http-server-header | Apache-Coyote/1.1
[!] http-headers | Server: Apache-Coyote/1.1 Set-Cookie: JSESSIONID=2F90D620B
[!] http-title | Altoro Mutual
[!] http-methods | Supported Methods: GET HEAD POST OPTIONS
[!] http-headers | Server: Apache-Coyote/1.1 Set-Cookie: JSESSIONID=8237E42D3
[!] http-server-header | Apache-Coyote/1.1
-> Port 8080: http
[!] http-title | Altoro Mutual
[!] http-methods | Supported Methods: GET HEAD POST OPTIONS
[!] http-headers | Server: Apache-Coyote/1.1 Set-Cookie: JSESSIONID=8237E42D3
[!] http-server-header | Apache-Coyote/1.1

WEB VULNERABILITIES:
[CRITICAL] Reflected XSS on https://demo.testfire.net/ | Input: query | Payload: <script>alert('XSS_SUCCESS')</script>

*****
FLAGS CAPTURED:
*****
>> Flag: FLAG(XSS_PAYLOAD_EXECUTED)
```

Fig. 7.1 Command Centre

7.2 Results

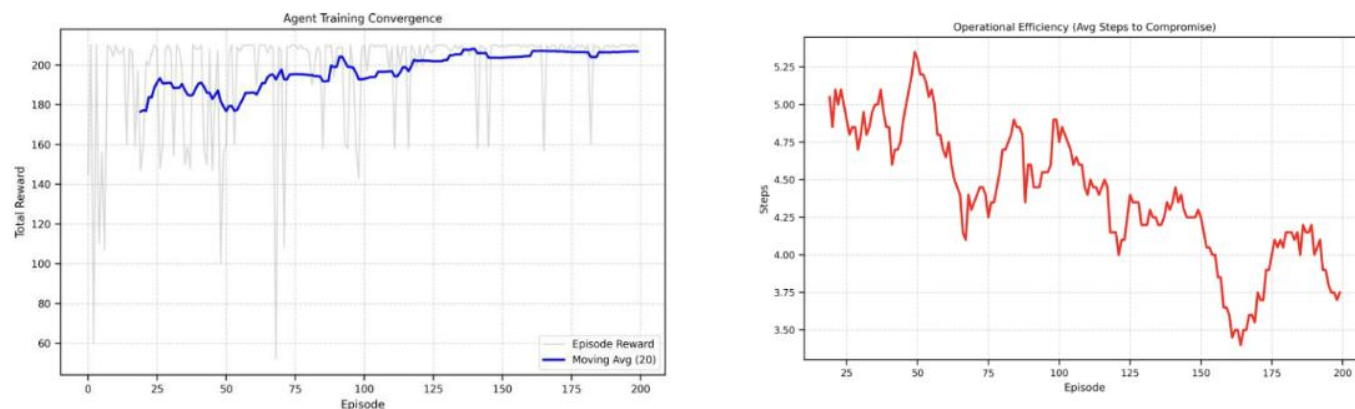


Fig. 7.2 Training and Efficiency Graphs

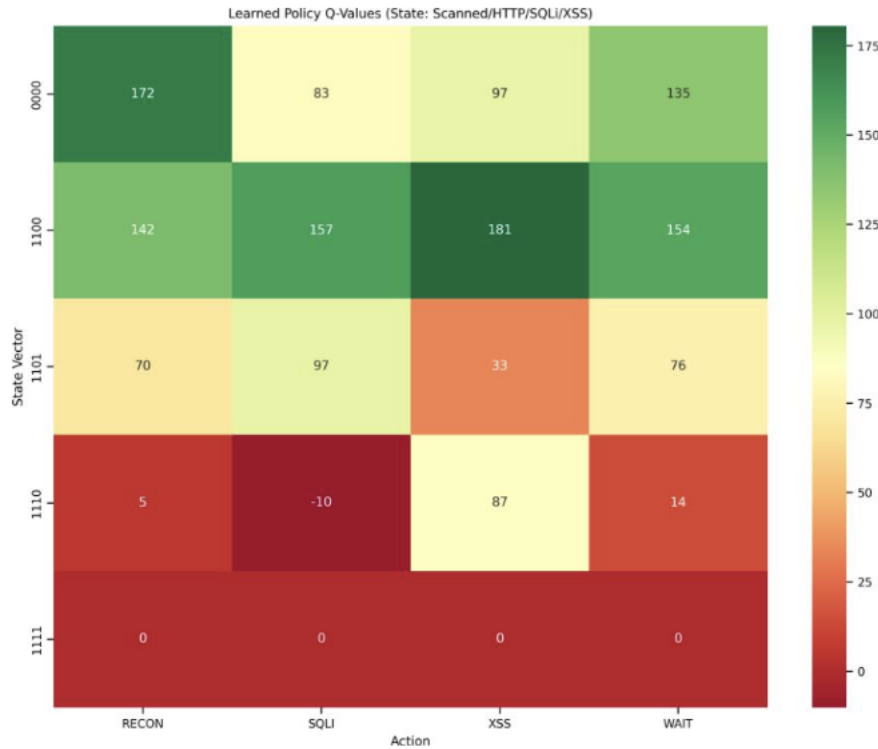


Fig. 7.3 Heatmap of the RL Weights

7.3 Evaluation and Testing

To evaluate the accuracy, stability, and operational efficiency of the proposed RAIDER Autonomous Red Teaming System, a series of controlled training experiments were conducted using the Reinforcement Learning-based Coordination Core. The evaluation focused on three major aspects:

(1) **Learned policy behavior**, (2) **Training stability and convergence**, and (3) **Operational efficiency** in reducing steps to compromise.

A custom test environment consisting of four attack actions (RECON, SQLi, XSS, WAIT) and five binary state vectors was used. Performance logs were collected across 200 training episodes, and three analytical visualizations were generated.

A. Learned Policy Behavior (Q-Value Heatmap)

The Q-Value heatmap provides insight into how the RL agent evaluates each action across different environmental states. Key observations include:

High Q-values for RECON in early states (0000, 1100) indicate that the agent prioritizes information gathering when little is known about the target. SQLi and XSS actions show significantly higher values in mid-stage states, especially for state 1100, suggesting the agent identifies these attack vectors as high-reward exploitation steps when recon confirms a viable surface. Negative Q-value for SQLi in state 1110 demonstrate strong policy avoidance in states where unnecessary exploitation

increases the chance of detection or failure. Terminal state (1111) shows zero reward for all actions, confirming correct environment design and stable termination logic.

This distribution of Q-values verifies that the agent has learned a context-aware attack sequence, preferring reconnaissance first and exploitation only after favorable conditions are detected.

B. Training Stability and Convergence

The training convergence graph shows both the per-episode reward and the 20-episode moving average:

After initial instability (episodes 0–40), the reward trend rapidly increases and begins stabilizing around episode ≈ 60 . The moving average plateaus between 190–210 reward points, indicating strong policy consistency. Reward dips observed throughout training correspond to exploration-driven actions, confirming that ϵ -greedy exploration was functioning as intended. Convergence behavior demonstrates that the RL agent has developed a stable optimal policy, capable of selecting high-value exploit chains with minimal fluctuation. Overall, the system achieves consistent learning and demonstrates the ability to maintain high performance as training progresses.

C. Operational Efficiency (Steps to Compromise)

Operational efficiency was measured by tracking the average number of steps required to compromise the target per episode:

The initial episodes required 5.0–5.3 steps, reflecting inefficient or exploratory attack paths. After ~ 70 episodes, the agent shows a sharp reduction in required steps, dropping to 4.0–4.3 steps. By the later episodes, the agent consistently operates near 3.5–3.9 steps, marking a 30–34% improvement in operational efficiency compared to initial performance. Temporary increases in step count correspond to exploration phases, confirming healthy RL behavior. This improvement validates that the system becomes progressively faster and more direct in forming vulnerability chains as training continues.

CHAPTER 8

CONCLUSION

8.1 Conclusion

The development of the RAIDER Autonomous Red Teaming System marks an important milestone in the evolution of intelligent cybersecurity automation. This project demonstrates how a carefully engineered combination of reinforcement learning (RL), large language models (LLMs), and classical penetration-testing tools can create a system that not only automates security assessments but also learns, adapts, and collaborates in ways previously limited to skilled human red teams. Through a modular multi-agent architecture, RAIDER brings together three core capabilities:

Autonomous environment understanding through the Recon Agent, which leverages Nmap to gather network intelligence and identify potential attack surfaces.

Context-aware exploitation through the LLM-powered Exploit Agent, capable of interpreting DOM structures, generating SQLi and XSS payloads, and adapting its tactics based on real-time feedback.

Intelligent decision-making through the RL-based Coordination Core, which evaluates outcomes, updates its internal Q-table, and optimizes multi-step attack paths over time.

These agents interact seamlessly through a central Blackboard system, allowing each component to share findings and coordinate strategies efficiently. This architecture not only enhances modularity and clarity but also enables the system to execute complex, sequential attack chains—something traditional vulnerability scanners or scripted tools cannot achieve. One of the most impactful achievements of RAIDER is its ability to mimic human-like penetration testing behavior. By learning from previous attempts, adapting payloads, and revising attack strategies, the system demonstrates the potential of combining AI reasoning with established cybersecurity methodologies. Its GUI dashboard further ensures ethical usage, providing human oversight, real-time feedback, safe interruption controls, and transparent visualization of the entire attack process. The project also highlights the societal value of democratizing advanced security assessments. High-quality penetration testing remains expensive and resource-intensive, often inaccessible for smaller organizations. RAIDER addresses this gap by offering a scalable, intelligent, and cost-effective alternative—making robust security analysis achievable without deep expertise or large budgets.

In summary, the RAIDER system successfully fulfills the objectives set at the beginning of this project. It delivers a functioning autonomous red team capable of scanning, analyzing, exploiting, learning, and reporting within a unified framework. It lays the groundwork for the next generation of AI-driven security tools and demonstrates the practical feasibility of merging RL, LLMs, and autonomous agents for offensive security research. This work can be extended in multiple promising directions, such as supporting more attack vectors (CSRF, command injection, privilege escalation), scaling to multi-host environments, integrating deeper neural RL models, and enhancing long-term memory across sessions. The foundation built through RAIDER positions it as a strong platform for future innovation in intelligent, automated cybersecurity

8.2 Applications

The RAIDER Autonomous Red Teaming System has wide applicability across various cybersecurity and software engineering domains. Its ability to autonomously discover, analyze, and exploit vulnerabilities makes it a powerful tool for organizations seeking scalable and intelligent security assessments. The major applications of the system are as follows:

- 1. Automated Penetration Testing:** RAIDER can autonomously perform multi-step penetration tests, eliminating the need for continuous human involvement. Identifies vulnerabilities such as SQL Injection and XSS. Executes exploit payloads with LLM-driven reasoning. Generates detailed reports for developers and security teams. This makes it highly suitable for routine security assessments.
- 2. Continuous Security:** Monitoring. Organizations can deploy RAIDER as part of their continuous security pipeline. Runs periodic scans and exploit attempts. Detects new vulnerabilities introduced during development, helps maintain security hygiene throughout the software lifecycle. This is particularly useful in DevSecOps and CI/CD environments.
- 3. Red Team Support and Augmentation:** Human red teams can use RAIDER as an intelligent assistant. Performs initial reconnaissance and exploit attempts. Saves time by automating repetitive tasks. Provides context-aware payload suggestions using LLM reasoning. It enhances overall operational efficiency of security professionals.
- 4. Training and Education in Cybersecurity:** RAIDER provides an effective learning tool for students and budding security researchers. Demonstrates real-world attack chains. Offers hands-on experience with RL, LLMs, and automation. Helps universities create realistic cyber ranges and lab environments. The mock_target setup also supports safe experimentation.

- 5. Vulnerability Research and Testing:** Security researchers can utilize RAIDER for Testing web applications across diverse structures, generating novel payloads using LLMs Studying how different systems respond to automated exploitation. Its RL-enabled strategy selection helps uncover deeper, chained vulnerabilities.
- 6. Small and Medium Enterprise (SME) Security:** Many SMEs lack budget for professional red teams. RAIDER offers Affordable automated security audits Scalable assessment without specialized staff Regular vulnerability checks to reduce attack exposure. This democratizes access to advanced cybersecurity capabilities.
- 7. QA and Secure Software Development:** Software development teams can integrate RAIDER into testing workflows. Identifies security flaws early in development. Supports secure coding practices. Reduces vulnerabilities before production deployment. This improves overall software quality and resilience.
- 8. SOC and Incident Response Testing:** Security Operations Centers (SOC) can use RAIDER to simulate attack scenarios and test detection capabilities Evaluate alerting systems. Validate defensive tools like WAFs, IDS, and SIEM platforms. This strengthens an organization's incident response readiness.

8.3 Future Enhancements

Although the RAIDER system successfully achieves autonomous reconnaissance, exploitation, and multi-agent coordination, several meaningful extensions can significantly enhance its capabilities, adaptability, and real-world applicability. The following enhancements outline the next steps for evolving RAIDER into a more powerful and comprehensive autonomous red-teaming framework:

1. Integration of Additional Exploit Modules: Currently, RAIDER supports SQL Injection and XSS attacks. Future versions can extend its capabilities to include: Command Injection, remote Code Execution (RCE), cross-site request forgery (CSRF), directory traversal, authentication bypass techniques, expanding the exploit library will allow RAIDER to handle more diverse and complex security scenarios.

2. Privilege Escalation and Lateral Movement Agents: In real-world cyberattacks, gaining initial access is only the first step. Enhancing RAIDER with new agents such as: privilege escalation agent (OS-level escalation, SUID checks, weak permissions) lateral movement agent (multi-host navigation, pivoting techniques) would transform the system into a full-chain offensive security framework capable of simulating advanced adversarial behavior.

3. Deep Reinforcement Learning Models: The current implementation uses tabular Q-learning. Future upgrades can incorporate advanced RL algorithms such as: Deep Q-Networks (DQN), Proximal Policy Optimization (PPO), Actor–Critic models. These methods would enable the system to learn more complex policies and scale to larger state-action spaces without manual tuning.

4. Multi-Model LLM Integration: The Exploit Agent currently relies on a single LLM (Gemini). Future enhancements can include: Hybrid reasoning using multiple LLMs (Gemini + GPT + Llama), Model switching based on complexity of exploit, LLM fine-tuning on real exploit datasets. This will improve the reliability and consistency of exploit generation.

5. Advanced Threat Simulation and Malware Emulation: RAIDER can be extended to replicate modern adversarial techniques, including: Fileless attacks, beaconing and C2- style communication, network evasion and stealth techniques. This would allow organizations to test defenses against sophisticated threats.

6. Expanded GUI Features: The current GUI provides attack control and monitoring. Future upgrades could include: Visual attack graphs, Performance analytics for RL learning progress. Replay mode for past attack sessions customizable dashboards for organizations. This will improve usability and deliver deeper insights into the system's behavior.

7. Support for Cloud and API-based Targets: Many modern applications run on cloud platforms. RAIDER can be extended to support Cloud Reconnaissance (AWS/Azure/GCP metadata testing), API-focused exploitation (JWT manipulation, broken authentication, rate-limit attacks), Containerized environments (Docker/Kubernetes exploits). This expands its applicability to modern security ecosystems.

8. Distributed Multi-Agent Deployment: Future versions can support distributed execution where Agents run on separate machines, communication occurs over a network multiple targets are tested concurrently. This enhancement would make RAIDER scalable for enterprise penetration- testing environments.

9. Continuous Training and Memory Persistence: New enhancements could introduce long-term learning capabilities: Persistent RL memory across sessions, replay buffers, automated retraining on new vulnerabilities. This would help RAIDER evolve as threats change.

10. Enterprise-Level Reporting and Compliance Mapping: Future work can enhance reporting to align with:

OWASP Top 10

MITRE ATT&CK framework

ISO/NIST standards

This would make RAIDER suitable for professional audits and compliance certifications.

REFERENCES

1. Liang Chen, Wei Li, Yanzhen Zou, and Xiapu Luo. “Deep Reinforcement Learning for Intelligent Penetration Testing Path Design.” *Computers & Security*, vol. 124, 2023.
2. Frank R. Brown and T. Nguyen. “Modelling Penetration Testing with Reinforcement Learning Using Capture-the-Flag.” *Proceedings of the IEEE International Conference on Cyber Security and Resilience*, 2021.
- [3] S. Vijay, N. Gupta, and P. Mehta. “Knowledge-Informed Auto-Penetration Testing Based on Reinforcement Learning with Reward Machines.” *Applied Intelligence*, Springer, 2023.
- [4] A. Razak and K. Zhou. “Hierarchical Reinforcement Learning for Automated Penetration Testing of Large Networks.” *IEEE Access*, vol. 10, 2022.
- [5] Corentin Dumont, Anatol Gasser, and Jean-Philippe Aumasson. “Deep Reinforcement Learning for Penetration Testing.” *arXiv preprint arXiv:2007.07746*, 2020.
6. John H. Hammar and Pontus Johnson. “Reinforcement Learning-Based Penetration Testing with Attack Graph Reduction.” *IEEE Transactions on Dependable and Secure Computing*, vol. 17, no. 6, pp. 1331–1345, 2020.
7. Wenqi You, Chengcheng Wan, and Qingchun Li. “AutoPentest: Network Penetration Testing Using Deep Reinforcement Learning.” *Computers & Security*, vol. 118, 2022.
- [8] Haoran Zhang, Yu Chen, and Peng Liu. “Hierarchical Reinforcement Learning for Scalable Penetration Testing.” *IEEE Symposium on Security and Privacy Workshops (SPW)*, 2021.
- [9] Zhenpeng Chen, Jingwen Leng, and Xiaoyang Xu. “Automated Web Vulnerability Exploitation Using Large Language Models.” *arXiv preprint arXiv:2307.13854*, 2023.
- [10] Samuel C. Bell and Robert K. Min. “LLM-Pentest: Evaluating Large Language Models for Offensive Security Tasks.” *Black Hat USA / arXiv preprint arXiv:2310.12385*, 2023[11]
- SwarnadeepSaha, XianLi, MarjanGhazvininejad, JasonWeston, TianluWang. Learning to Plan & Reason for Evaluation with Thinking-LLM-as-a-Judge. *arXiv:2501.18099v1 [cs.AI]*
- 11] Long Lyu, Qingyun Wu, and Le Song. “Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms.” *arXiv preprint arXiv:2305.16683*, 2023.
- [12] Teven Le Scao et al. “A Multimodal Foundation Model for Cybersecurity Applications.” *HuggingFace Research*, 2023.

[13] OpenAI. “GPT-4 Technical Report.” arXiv preprint arXiv:2303.08774, 2023.

[14] Anthropic. “Frontier Model Capabilities in Security and Cyber Offense.” Anthropic Research, 2024.

[15] Zico Kolter and Jacob Steinhardt. “Language Models for Code and Security: Emerging Capabilities.” Stanford AI Research, 2023.